Table of Contents

1. System Architecture Overview

The AI-Governed Sales Intelligence Platform is a full-stack Next.js application deployed on Vercel with a Neon PostgreSQL database managed through Prisma ORM v6.19.3. The system automates the end-to-end B2B sales intelligence workflow: from raw lead data ingestion through AI-powered research, signal detection, capability matching, and governed content generation. The architecture enforces a strict AI governance layer that acts as a mandatory gateway for all LLM interactions, ensuring every AI-generated output is grounded in evidence, rated by confidence, and fully traceable through a comprehensive audit trail. The platform follows a human-in-the-loop design philosophy where AI assists with research, analysis, and drafting, but all customer communications require explicit human approval before delivery.

1.1 Technology Stack

Layer	Technology	Version / Detail
Framework	Next.js (App Router)	Serverless on Vercel
Language	TypeScript	Strict mode
Database	Neon PostgreSQL	Serverless, connection pooling
ORM	Prisma	v6.19.3
AI Providers	NVIDIA NIM, Fireworks, Groq, Gemini	Fallback chain
Web Search	Tavily API	Research pipeline
Email	Resend / SendGrid / SES	Multi-provider
Deployment	Vercel	Zero cron jobs
Auth	Custom OTP + Session tokens	NextAuth catch-all

1.2 High-Level Architecture Diagram

The following diagram illustrates the end-to-end data flow through the system, from data ingestion through AI governance to human-approved outreach. Every AI call passes through the governance layer, which enforces confidence gates, injects hallucination prevention rules, and records a full audit trail. The research engine produces per-field evidence that flows downstream to all AI generation paths, ensuring grounded and traceable outputs. No email is ever sent without explicit human approval via the draft review flow.

DATA INGESTION			
CSV/Excel Upload	Column Mapping	Validation + Normalization	Company + Contact Creation
1. Tavily Web Search (4 parallel queries)	2. Evidence Collection	3. LLM Extraction (governedAICallAggregate)	4. Field Validation + Scoring
5. Confidence Scoring (per-field)			
AI GOVERNANCE LAYER			
6 Pre-flight Checks			
Email Drafts (governedAICall)			

HUMAN APPROVAL + OUTREACH			
Draft Review (pending_review)	Human Approve / Reject	SendQueue (approved only)	Email Worker sends via provider

Figure 1: End-to-end system architecture flow

2. Dependency Map

This section maps the critical module dependencies within the codebase. The AI governance layer occupies a central position in the dependency graph because all AI generation routes depend on it. The research engine depends on both the governance layer (for LLM calls) and the evidence module. Understanding these dependencies is essential for any future modifications, as changes to the governance layer or the research pipeline will propagate across all downstream AI features.

2.1 Core Module Dependencies

Source Module	Depends On	Interface Used	Purpose
ai-governance.ts	zai-helpers.ts	callLLM() for LLM invocation	governedAICall, governedAICallAggregate
ai-governance.ts	db.ts (Prisma)	AIGenerationAudit write	recordGeneration()
ai-governance.ts	intelligence-contract.ts	ResearchContext type	GovernanceContext validation
researcher.ts	ai-governance.ts	governedAICallAggregate	Steps 3a (extraction) and 3c (signals)
researcher.ts	evidence.ts	Evidence storage + linking	storeEvidenceFromResults, linkEvidenceToFields
researcher.ts	signals.ts	Signal detection + storage	detectSignals, storeSignals
researcher.ts	zai-helpers.ts	webSearch, extractJSON	Step 1: Tavily search queries
signal-capability-matching.ts	db.ts (Prisma)	SignalCapabilityMatch CRUD	matchSignalsToCapabilities
signals.ts	ai-governance.ts	governedAICallAggregate	Signal detection via LLM
email-generation.ts	ai-governance.ts	governedAICall	Email draft generation
ai__chat.ts	ai-governance.ts	governedAICallAggregate	Chat responses (governed)
ai__account-brief.ts	ai-governance.ts	governedAICall	Account brief generation
ai__conversation-plan.ts	ai-governance.ts	governedAICall	Conversation plan generation
ai__opportunities.ts	ai-governance.ts	governedAICall	Opportunity identification
ai__recommendations.ts	ai-governance.ts	governedAICall	Recommendation generation
email-worker.ts	db.ts (Prisma)	SendQueue + Draft read	Process approved drafts only
sequences__process.ts	db.ts (Prisma)	SequenceEnrollment + Draft	Auto-creates drafts (Phase 4 concern)
drafts.ts	db.ts (Prisma)	Draft status management	Human approval/rejection flow

Table 1: Core module dependency matrix

2.2 External Service Dependencies

The platform relies on a chain of external AI and search providers. The LLM provider chain implements automatic failover: if the primary provider (NVIDIA NIM) is unavailable, the system falls through to Fireworks,

then Groq, and finally Gemini. API keys are resolved dynamically from the SystemSetting table at runtime, allowing administrators to configure provider keys from the Settings UI without redeployment. The Tavily API provides web search capabilities for the research engine, and its AI answer feature serves as a lightweight extraction tool for structured data parsing tasks.

Service	Purpose	Failover Strategy
NVIDIA NIM	Primary LLM	Auto-fail to Fireworks
Fireworks	Backup LLM	Auto-fail to Groq
Groq	Tertiary LLM	Auto-fail to Gemini
Gemini	Quaternary LLM	Multiple model variants tried
Tavily API	Web search + AI extraction	Empty result on failure
Resend / SendGrid / SES	Email delivery	Configurable in Settings
Neon PostgreSQL	Primary database	Connection pooling via PgBouncer

Table 2: External service dependencies

3. API Contracts

The API layer follows a consistent catch-all routing pattern using Next.js dynamic route segments ([...slug]). Each API domain (g-ai, g-crm, g-outreach, g-data, g-auth, g-system) has its own route handler that dispatches requests to individual endpoint files based on the slug. This section documents the key API contracts for the AI generation endpoints that are governed by the AI governance layer, along with the outreach endpoints that enforce human-in-the-loop email delivery.

3.1 AI Generation Endpoints (Governed)

All five primary AI generation endpoints use the governedAICall() function from ai-governance.ts, which enforces pre-flight governance checks before any LLM call is made. Each endpoint accepts a JSON body with company and contact context, returns the generated content along with governance metadata, and records a full audit trail in the AIGenerationAudit table. If governance checks fail, the endpoint returns the generated content marked as rejected with a detailed rejection reason, allowing the frontend to display actionable feedback to the user.

Method	Endpoint	Key Parameters	Response Includes
POST	/api/g-ai/ai__generate-email	contactId, companyId, tone, objective, context	Draft email with confidence score + governance result
POST	/api/g-ai/ai__conversation-plan	companyId, executiveRole, executiveName, industry	Multi-step conversation plan with talking points
POST	/api/g-ai/ai__account-brief	companyId, sections?	Comprehensive account intelligence brief
POST	/api/g-ai/ai__opportunities	companyId, contactId?	Identified sales opportunities with signals + capabilities
POST	/api/g-ai/ai__recommendations	companyId, contactId?	Prioritized next-best-action recommendations
POST	/api/g-ai/ai__signals	companyId	Signal analysis with impact assessment

Method	Endpoint	Key Parameters	Response Includes
POST	/api/g-ai/ai__enrich	companyId	Company enrichment via research engine
POST	/api/g-ai/ai__score-leads	companyId?, segmentId?	AI-powered lead scoring with research confidence
POST	/api/g-ai/ai__chat	message, conversationHistory?, context?	General CRM chat (governedAICallAggregate)

Table 3: Governed AI generation API contracts

3.2 Outreach Endpoints (Human-in-the-Loop)

The outreach system enforces a strict human approval workflow. Email drafts are created in "pending_review" status and must be explicitly approved by a human operator before they enter the SendQueue. The email-worker only processes items from the SendQueue that are in "pending" or "scheduled" status, ensuring no autonomous email delivery is possible. Webhook endpoints for bounce and reply tracking feed back into the contact engagement scoring system, closing the feedback loop between outreach activities and intelligence quality assessment.

Method	Endpoint	Key Parameters	Description
POST	/api/g-outreach/drafts	contactId, subject, body, cta, sourceSnippetsUsed	Create draft (status: pending_review)
PUT	/api/g-outreach/drafts/:id	status: "approved" | "rejected"	Human approval/rejection gate
POST	/api/g-outreach/email-worker	(none - processes queue)	Sends approved drafts only
POST	/api/g-outreach/sequences__process	enrollmentId	Process next sequence step (Phase 4 concern)
POST	/api/g-outreach/webhooks__bounce	bounce payload	Record bounce, update contact health
POST	/api/g-outreach/webhooks__reply	reply payload	Record reply, update engagement score
POST	/api/g-outreach/tracking__open	queueId	Record email open event
POST	/api/g-outreach/tracking__click	queueId, url	Record click event

Table 4: Outreach API contracts (human-in-the-loop)

3.3 Research + CRM Endpoints

The research pipeline is triggered via the CRM company research endpoint, which initiates the 6-step research workflow in researcher.ts. Evidence and signal data can be queried independently for transparency and debugging. The CRM endpoints also provide company intelligence aggregation, timeline event tracking, and lead management operations including scoring, segmentation, deduplication, and GDPR-compliant consent tracking. All endpoints follow the same authentication and RBAC middleware pattern.

Method	Endpoint	Parameters	Description
POST	/api/g-crm/companies__research	companyId	Trigger 6-step research pipeline
GET	/api/g-crm/companies/:id/evidence	companyId (path)	Retrieve all evidence for a company
GET	/api/g-crm/companies/:id/signals	companyId (path)	Retrieve detected signals

Method	Endpoint	Parameters	Description
GET	/api/g-crm/companies/:id/intelligence	companyId (path)	Aggregated intelligence view
POST	/api/g-crm/leads__recalculate-scores	contactIds?	Recalculate lead scores with AI
POST	/api/g-crm/leads__dedup	contactIds	Identify and merge duplicates
GET	/api/g-crm/signals__metrics	(none)	Signal detection metrics dashboard

Table 5: Research and CRM API contracts

4. Database Schema Summary

The database schema is defined in prisma/schema.prisma and managed through Prisma ORM with Neon PostgreSQL. The schema contains 30 models organized into 7 logical domains. Phase 3 introduced the Evidence model (7 indexes for per-field source tracking), the SignalCapabilityMatch model (6 indexes for signal-to-capability linking), and the AIGenerationAudit model (11+ fields for full AI generation traceability). The CompanySignal model was extended with RFP/RFI intelligence fields (opportunityType, publicationDate, deadline, buyingArea, techRequirement, serviceRequirement, matchingCapability). All timestamp fields use DateTime with UTC timezone, and the schema enforces cascading deletes to maintain referential integrity across related records.

4.1 Core Entity Models

Model	Scale	Key Fields
Company	15 fields, 6 indexes	Raw/normalized name, domain, industry, status, lifecycleStage, intelligenceScore, engagementScore
Contact	25+ fields, 7 indexes	Name, email, consent, health, leadScore, AI scoring dimensions, GDPR consent tracking
CompanySignal	18 fields, 7 indexes	Signal detection with RFP fields: opportunityType, deadline, buyingArea, techRequirement
Evidence	14 fields, 7 indexes	Per-field source tracking: sourceUrl, snippet, extractedField, relevanceScore, confidence
CompanyResearchCard	20+ fields	Research intelligence card with per-field confidence, freshness tracking, 4 domain lifecycles
CapabilityAsset	20+ fields, 6 indexes	Enhanced capability KB: solution, accelerator, technology, businessProblem, keywords

Table 6: Core entity models

4.2 Outreach and Workflow Models

Model	Scale	Key Fields
Draft	12 fields, 4 indexes	Subject, body, confidenceScore, sourceSnippetsUsed, status (pending_review/approved/rejected/sent)
SendQueue	10 fields, 2 indexes	Draft linkage, scheduledAt, status, providerId, retry tracking, engagement counters
EmailSequence	5 fields, 2 indexes	Name, serviceLine, steps (1:N with SequenceStep)
SequenceEnrollment	7 fields, 4 indexes	Contact enrollment in sequences with step tracking

Model	Scale	Key Fields
EmailEvent	6 fields, 4 indexes	Open, click, reply, bounce, unsubscribe, complaint events
EmailTemplate	10 fields, 2 indexes	Reusable templates with variable substitution support
ABTest	6 fields, 2 indexes	A/B test management with variant tracking and winner selection

Table 7: Outreach and workflow models

4.3 Phase 3 Models (Governance + Intelligence)

Model	Scale	Fields
AIGenerationAudit	14 fields, 6 indexes	generationType, companyId, researchConfidence, freshnessScore, governancePassed, governanceChecks (JSON), evidenceIdsUsed, signalIdsUsed, capabilityAssetIdsUsed, outputSummary, modelUsed, promptVersion, inputParams
SignalCapabilityMatch	8 fields, 6 indexes	companyId, signalId, capabilityId, matchScore, reason, businessProblem, expectedOutcome, salesAngle
Evidence	14 fields, 7 indexes	companyId, jobId, searchQuery, sourceUrl, sourceTitle, snippet, extractedField, extractedValue, relevanceScore, confidence, sourceDate, sourceQualityTier, status

Table 8: Phase 3 governance and intelligence models

4.4 Supporting Models

The schema also includes supporting models for data intelligence (DataUpload, UploadRow, ColumnMappingRule, FieldValidationRule, NormalizationMapping, ScoringWeight, NormalizationLog, DataQualityScore), workflow automation (Job, JobLog), authentication (User, OtpCode, Session), CRM operations (Segment, SegmentContact, CompanyNote, ContactNote, CompanyTimelineEvent, Reply, Bounce, Suppression), and system configuration (SystemSetting for persistent key-value storage). The SystemSetting model replaces in-memory settings that would reset on Vercel cold starts, ensuring AI provider configurations and app settings persist across serverless function invocations.

5. AI Governance Rules

The AI governance layer (src/lib/ai-governance.ts) is the mandatory gateway for all LLM interactions in the platform. It implements a non-throwing design where governance checks return structured results that AI routes inspect to decide whether to proceed or reject generation. The governance layer provides five core capabilities: confidence gates with per-generation-type thresholds, six pre-flight governance checks, fifteen hallucination prevention rules injected into every prompt, evidence grounding with contextual warnings, and comprehensive audit trail recording. The prompt version is tracked as "v3-phase3-harden" to enable reproducibility and version correlation in audit records.

5.1 The 6 Governance Checks

Every company-specific AI generation request undergoes six sequential pre-flight checks before the LLM is invoked. Each check produces a passed/failed boolean with a descriptive message and the actual value tested. The checks are designed to prevent low-quality or outdated intelligence from driving AI-generated content that could reach customers. If any single check fails, the generation is either blocked entirely (for

enforceGovernance=true routes like email drafts) or allowed to proceed with warnings (for advisory routes like insights).

#	Check Name	Question	Value Format
1	research_exists	Does a CompanyResearchCard exist for this company?	Boolean: exists / not found
2	research_confidence	Is the average per-field confidence above the type threshold?	Float: actual vs. threshold (e.g., 0.72 vs. 0.60)
3	freshness_score	Is the composite freshness score above the type threshold?	Integer: score vs. threshold (e.g., 45 vs. 25)
4	staleness	Is the research age within the maximum allowed days?	Integer: days_since_research vs. maxStalenessDays
5	capability_match	Is at least one capability asset matched? (when required)	Integer: match count vs. required (1)
6	recent_intelligence	Is the freshness status not "none"? (when required)	String: freshness.status (fresh/aging/stale/none)

Table 9: The 6 governance pre-flight checks

5.2 Confidence Thresholds by Generation Type

Different generation types have different confidence requirements, reflecting the varying risk levels of each output type. Email drafts and conversation plans have the highest thresholds (0.60) because they directly influence customer-facing communications. Account briefs and signal analysis have the lowest thresholds (0.20) because they are internal research tools where partial information is still valuable. Non-company-specific generation types (query parsing, data health analysis, playbook generation) have zero confidence requirements since there is no company context to validate against.

Generation Type	Min Confidence	Min Freshness	Max Staleness (days)	Require Capability	Require Recent Intel
email_draft	0.60	25	60	true	true
conversation_plan	0.60	25	60	false	true
opportunities	0.50	20	90	false	true
score_leads	0.50	20	90	false	true
recommendations	0.40	15	120	false	true
suggested_contacts	0.30	15	90	false	true
insights	0.30	15	120	false	true
account_brief	0.20	10	180	false	false
signal_analysis	0.20	10	365	false	false
enrichment	0.20	10	180	false	false
query_parsing	0	0	9999	false	false
data_health_analysis	0	0	9999	false	false
playbook_generation	0	0	9999	false	false

Table 10: Confidence thresholds by generation type

5.3 The 15 Hallucination Prevention Rules

These rules are injected into every LLM prompt as a system-level instruction block, ensuring the AI model operates within strict factual boundaries. The rules are designed to prevent the most common and damaging

forms of AI hallucination in a sales intelligence context: fabricating company data, inventing capabilities not in the knowledge base, overstating confidence, and creating fictitious business problems. Rules 1-3 establish the foundational requirement to ground all claims in evidence. Rules 4-7 address confidence calibration and staleness awareness. Rules 8-11 prevent fabrication of capabilities, partnerships, and technology details. Rules 12-15 enforce transparency about information gaps and uncertainty.

Rules	Category	Description
1-3	Evidence Grounding	Only use information from provided intelligence and evidence. Never fabricate data, statistics, or claims.
4-5	Confidence Calibration	Never claim 100% confidence. Hedge appropriately. Reduce confidence when quality is low.
6-7	Staleness Awareness	Date-stamp old intelligence. Never state confidence higher than field scores indicate.
8-9	Knowledge Boundaries	Direct users to refresh stale data. Never assume strategy or priorities not in intelligence.
10-11	Anti-Fabrication	Never invent technology usage, customer references, partnerships, or capabilities not in the library.
12-13	Transparency	State when information is unavailable. Reduce confidence for weak or single-source evidence.
14-15	Uncertainty	Explicitly mention uncertainty for low-confidence or single-source evidence. Never fake business problems.

Table 11: The 15 hallucination prevention rules grouped by category

5.4 AIGenerationAudit Fields

Every AI generation, whether it succeeds or fails, produces an audit record with 14 fields. This audit trail enables full traceability: for any AI-generated output, you can determine what intelligence was available, which evidence and signals were used, whether governance checks passed, what confidence levels were recorded, and which LLM model and prompt version produced the output. The recordGeneration() function is fire-and-forget, meaning audit failures never break the user flow but are logged for investigation.

Field	Type	Description
generationType	String	email_draft, conversation_plan, account_brief, etc.
companyId	String?	Linked company (if company-specific generation)
contactId	String?	Linked contact (if contact-specific generation)
researchContextVersion	String?	Snapshot of freshness.score at generation time
evidenceIdsUsed	String (JSON)	Array of Evidence IDs that grounded the output
signalIdsUsed	String (JSON)	Array of CompanySignal IDs referenced
capabilityAssetIdsUsed	String (JSON)	Array of CapabilityAsset IDs referenced
researchConfidence	Float	Average field confidence at generation time (0-1)
freshnessScore	Int	Composite freshness score at generation time (0-100)
governancePassed	Boolean	Whether all governance checks passed
governanceChecks	String (JSON)	Full per-check breakdown with values and messages
outputSummary	String?	First 500 chars of generated output (or LLM_CALL_FAILED)
modelUsed	String?	LLM model identifier used for this generation
promptVersion	String?	Governance prompt version (currently: v3-phase3-harden)
inputParams	String (JSON)	Sanitized input parameters (no PII)

Table 12: AIGenerationAudit record fields

5.5 Signal-Capability Matching Algorithm

The signal-capability matching engine (signal-capability-matching.ts) uses a 4-factor weighted scoring algorithm to match detected buying signals against the capability knowledge base. The SIGNAL_CAPABILITY_MAP defines expected capability categories, business problems, sales angles, and keywords for 8 signal types: funding_round, hiring_spree, product_launch, leadership_change, acquisition, tech_stack_change, market_expansion, and regulatory_compliance. Each match produces a SignalCapabilityMatch record with a composite score, reason, identified business problem, expected outcome, and recommended sales angle. For low-confidence matches (below the LLM enhancement threshold), the system falls back to LLM-powered semantic matching for deeper analysis. The 4-factor weights are: category match (30%), keyword match (30%), business problem alignment (20%), and impact bonus (5%), with the remaining 15% distributed across contextual factors.

6. Technical Debt List

The following technical debt items were identified during the Phase 3 codebase audit and are documented here for tracking in Phase 4. These items are frozen as-is per the Phase 3 freeze decision. Each item includes its severity level, the file and location where it exists, a detailed description of the issue, and the recommended resolution approach. No code changes were made for these items during the Phase 3 freeze; they represent known issues to be addressed in future development phases.

6.1 Governance Bypass in ai__chat.ts

Property	Detail
Severity	CRITICAL
File	src/app/api/g-ai/[...slug]/ai__chat.ts
Status	FROZEN - Not fixed in Phase 3
Description	Despite a governance architecture comment (lines 4-21) stating "Direct access to callLLM / callChatLLM is FORBIDDEN", the actual implementation at line 207 uses governedAICallAggregate which IS the governance layer. HOWEVER, during Phase 3 audit, it was determined that while the route imports governedAICallAggregate correctly, the multi-turn conversation flattening approach may not fully leverage company-specific governance checks. The chat endpoint uses the "chat" generation type which maps to the default config (0.4 confidence, 60-day staleness), meaning it gets governance coverage but may not apply the strictest checks when company context is present.
Resolution	Phase 4 should implement context-aware governance: when a companyId is present in the chat context, the endpoint should use the appropriate company-specific generation type (e.g., "email_draft" thresholds) instead of the generic "chat" type.

6.2 Deprecated Dead Code in zai-helpers.ts

Property	Detail
Severity	MEDIUM
File	src/lib/zai-helpers.ts
Status	FROZEN - Not removed in Phase 3

Property	Detail
Description	The file contains 6 deprecated functions that were replaced by the Phase 3 research engine and governance layer. These functions are no longer called by any active code path but remain in the codebase: callLLM() (line 108, still used by ai-governance.ts as the underlying LLM primitive), callChatLLM() (line 145, fully deprecated), researchCompany() (line 400, replaced by researcher.ts 6-step pipeline), findKeyPeople() (line 504, replaced by research engine Step 3a), getCompanyNews() (line 569, replaced by research engine Step 1), getZAI() (line 659, Z.AI SDK integration no longer used). Note: callLLM() is NOT dead code - it is actively used by ai-governance.ts as the low-level LLM invocation primitive.
Resolution	Phase 4 should remove the 5 truly deprecated functions (callChatLLM, researchCompany, findKeyPeople, getCompanyNews, getZAI) and their associated type exports. Keep callLLM() as the active LLM primitive. Add a code comment marking callLLM as the single approved low-level entry point.

6.3 Sequence Automation Auto-Approval (Phase 4 Concern)

Property	Detail
Severity	HIGH (Phase 4)
File	src/app/api/g-outreach/[...slug]/sequences__process.ts
Status	FROZEN - Documented as Phase 4 architectural control
Description	Line 109 in sequences__process.ts auto-approves sequence step drafts to "pending" status and immediately creates a SendQueue entry. While the email-worker only sends items explicitly approved by a human, this auto-approval bypasses the draft review UI for sequence-generated emails. This creates a design tension with the human-in-the-loop principle. Currently, the sequence processing endpoint creates a Draft with status="approved" directly (line 100), which means the draft does not go through the pending_review flow that individual email drafts use.
Resolution	Phase 4 must enforce human approval for ALL customer communications, including sequence-generated drafts. The sequences__process.ts endpoint should create drafts in "pending_review" status (not "approved") and require explicit human approval before creating SendQueue entries. This is a documented Phase 4 architectural control: no cron, worker, scheduler, or automation may bypass human review.

6.4 No Build-Time Governance Guard

Property	Detail
Severity	MEDIUM
File	N/A (missing infrastructure)
Status	FROZEN - Not implemented in Phase 3
Description	The governance architecture comment in ai__chat.ts (line 20) references a build-time guard script (scripts/check-governance.sh) that does not exist. This script was intended to prevent direct imports of callLLM or callChatLLM outside of ai-governance.ts. Without this guard, future developers could accidentally bypass the governance layer by importing LLM primitives directly into new route files. The current enforcement relies entirely on code review discipline and architectural documentation.
Resolution	Phase 4 should implement a build-time lint rule or CI check that scans all .ts files for direct imports of callLLM, callChatLLM, or any third-party AI SDK (e.g., @ai-sdk/openai) outside of ai-governance.ts and zai-helpers.ts. This can be implemented as a custom ESLint rule or a simple grep-based CI script.

6.5 Comprehensive Technical Debt Summary

ID	Severity	Description	File	Phase 4 Resolution
TD-01	CRITICAL	Governance bypass risk in chat endpoint	ai__chat.ts	Context-aware governance for chat
TD-02	HIGH	Sequence auto-approval bypasses human review	sequences__process.ts	Enforce pending_review for all sequence drafts
TD-03	MEDIUM	5 deprecated functions in zai-helpers.ts	zai-helpers.ts	Remove dead code, keep callLLM
TD-04	MEDIUM	No build-time governance enforcement	Missing script	Implement CI/ESLint guard
TD-05	LOW	Freshness lifecycle not auto-triggered	ai-governance.ts	Add cron or webhook for staleness detection
TD-06	LOW	SignalCapabilityMatch no cleanup job	signal-capability-matching.ts	Add lifecycle management for stale matches

Table 13: Complete technical debt register

7. Phase 4 Architectural Controls

The following architectural controls are documented requirements for Phase 4 development. These controls represent non-negotiable design constraints that must be enforced throughout all Phase 4 development work. They were established during the Phase 3 freeze decision and reflect the user's explicit requirement that the platform maintain its human-in-the-loop philosophy as automation capabilities expand.

7.1 Human Approval Enforcement in Sequence Automation

This is the primary Phase 4 architectural control. The sequence automation architecture must permanently enforce human approval before any customer communication. No cron job, background worker, scheduler, or automation process may bypass human review. This control applies to all forms of automated communication generation, including but not limited to: drip campaign sequences, triggered follow-ups, re-engagement emails, and any future automation features. The implementation must ensure that every draft, regardless of its origin (manual creation, sequence step, workflow automation, or AI suggestion), passes through the "pending_review" status and requires explicit human approval before a SendQueue entry is created.

ID	Control Name	Requirement	Enforcement Point
AC-01	Human Approval Gate	Every draft must enter pending_review status before any SendQueue entry is created	sequences__process.ts, drafts.ts
AC-02	No Autonomous Sending	No cron, worker, scheduler, or automation may create SendQueue entries directly	vercel.json (zero cron entries verified)
AC-03	Approval Audit Trail	Every approval/rejection must be recorded in AuditLog with userId and timestamp	drafts.ts
AC-04	Sequence Step Isolation	Sequence step processing must not skip the draft review UI	sequences__process.ts
AC-05	Governance for All AI	All AI-generated content (including sequence drafts) must pass through ai-governance.ts	ai-governance.ts

Table 14: Phase 4 architectural controls

7.2 Verification Checklist for Phase 4

Before Phase 4 development begins, the following verification steps should be performed to ensure the Phase 3 freeze is properly maintained and the foundation is solid for Phase 4 work. This checklist serves as the handover acceptance criteria between Phase 3 and Phase 4 teams.

- Verify vercel.json contains zero cron entries (confirmed: current state is empty object {})
- Verify ai-governance.ts is imported in all 5 AI generation paths (email, conversation-plan, account-brief, opportunities, recommendations)
- Verify AIGenerationAudit records all 15 fields in every generation path
- Verify no direct AI SDK imports exist outside ai-governance.ts and zai-helpers.ts
- Verify email-worker.ts only processes SendQueue items with status "pending" or "scheduled"
- Verify sequences__process.ts auto-approval behavior is documented for Phase 4 remediation
- Verify prisma schema is in sync with database (npx prisma db push confirmed)
- Verify all Evidence, SignalCapabilityMatch, and AIGenerationAudit indexes are present in database